Bird Behavior Classification using YOLOv8

Project Title

Bird Behavior Classification using YOLOv8 Deep Learning Model

---

Overview

Bird classification and behavior recognition are important tasks in wildlife monitoring, ecological research, and environmental conservation.

This project focuses on building a **deep learning-based bird classification system** that categorizes birds into three behavioral classes:

- **Fly**
- **Fly_Swim**
- **No_Fly**

The system uses a **YOLOv8 classification model** trained on a curated bird image dataset. The trained model can also be used for **real-time bird behavior detection using a webcam**.

The objective of this project is to demonstrate how **computer vision and deep learning can be applied to bird monitoring systems**.

Dataset

Two datasets were used in this project.

1️⃣ **CUB-200-2011 Dataset**

The **CUB-200-2011 (Caltech-UCSD Birds)** dataset is a widely used dataset for bird classification tasks.

Dataset details

- Total Images: ~11,788
- Bird Species: 200
- Image Annotations:
    - Bounding Boxes
    - Class Labels
    - Image IDs
    - Attributes

For this project, the dataset was **reorganized into behavioral categories**:

- Fly
- Fly_Swim
- No_Fly

2️⃣ Birds Images Dataset

An additional bird image dataset was used to enrich the **No_Fly class**.

Dataset description

This dataset contains a **diverse collection of bird images captured in natural habitats**, showing different species, environments, and bird behaviors.

Source: Pixabay

License: Apache 2.0

This dataset includes images such as:

- Owls
- Forest birds
- Ground birds
- Birds resting on trees

These images help improve **model generalization for non-flying bird behavior**.

Installation

Clone the repository and install the required dependencies.

```
git clone https://github.com/yourusername/bird-classification-yolov8.git
cd bird-classification-yolov8
```

Install required libraries:

```
pip install ultralytics
pip install opencv-python
pip install torch
```

Usage

1️⃣ Train the Model

```
python train_model.py
```

Example training parameters:

- Epochs: **20 / 32 / 64**

- Image Size: **224**
- Batch Size: **16**

---

2 Run Camera Demo

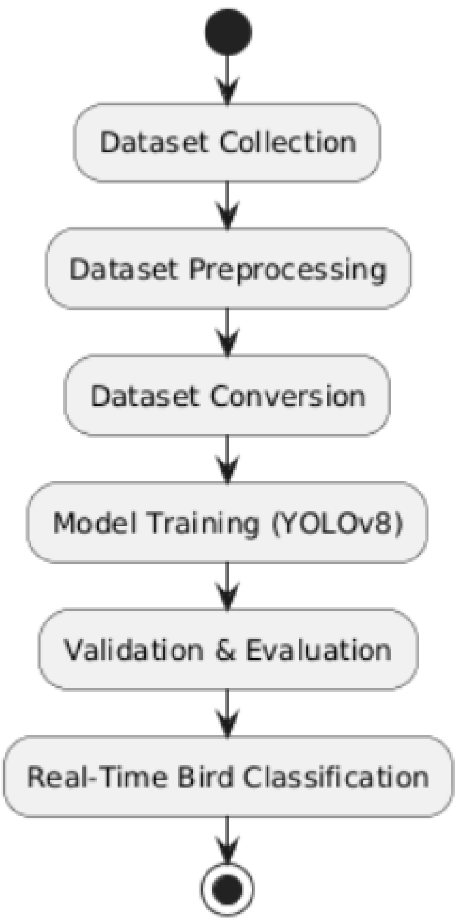
python camera_test.py

This will open a **webcam window** and classify bird behavior in real time.

Example output:

Bird Category: Fly
Bird Category: Fly_Swim
Bird Category: No_Fly

Methodology



Key Steps

1. Dataset collection from multiple sources
2. Data cleaning and filtering

3. Dataset conversion to YOLOv8 classification format
4. Model training using YOLOv8
5. Validation using unseen images
6. Real-time inference using webcam

Models Used

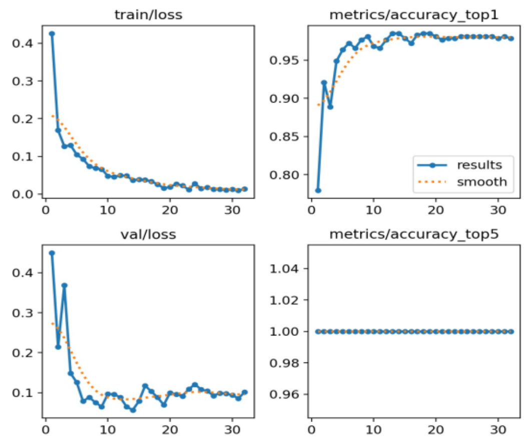
The following deep learning model was used:

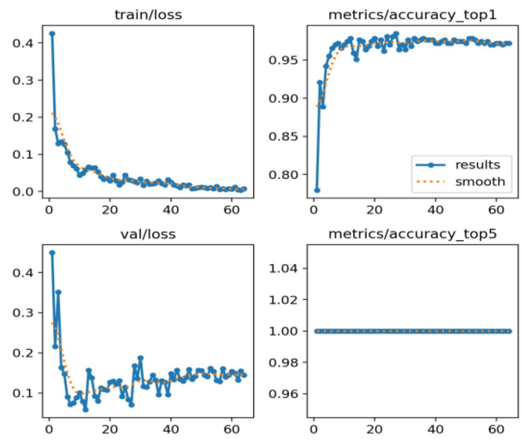
YOLOv8 Classification Model

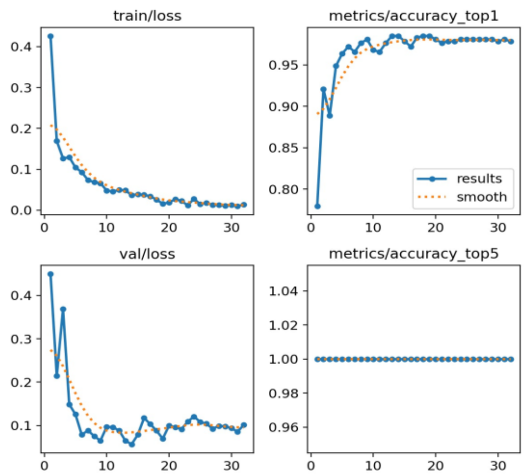
Framework: **Ultralytics YOLOv8**

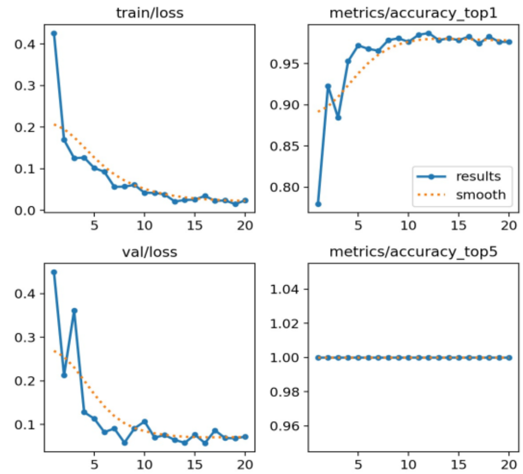
Advantages:

- Fast training
- High accuracy
- Efficient real-time inference









Model configuration:

Parameter	Value
Model	YOLOv8n-cls
Epochs	20 / 32 / 64
Image Size	224
Optimizer	AdamW
Framework	PyTorch

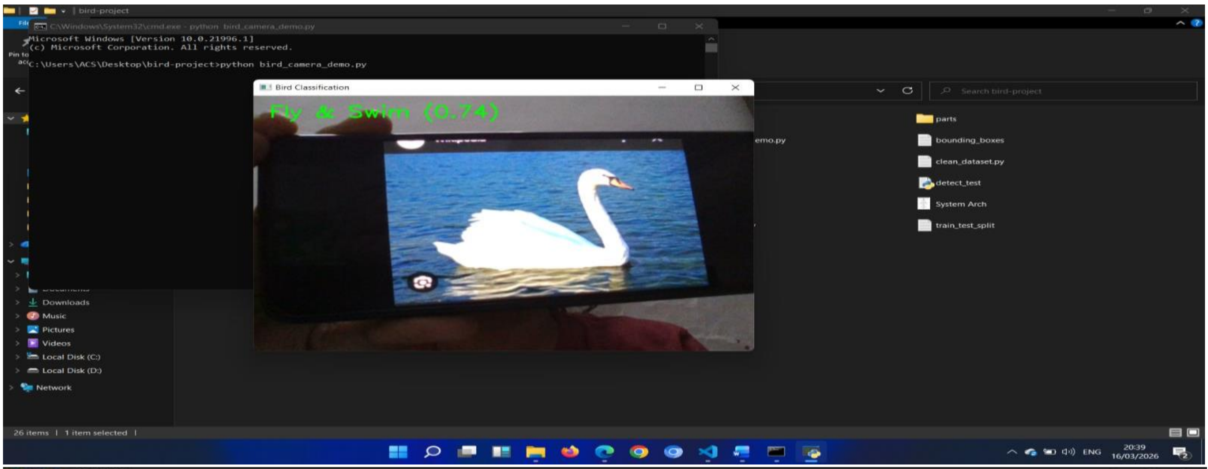
Results

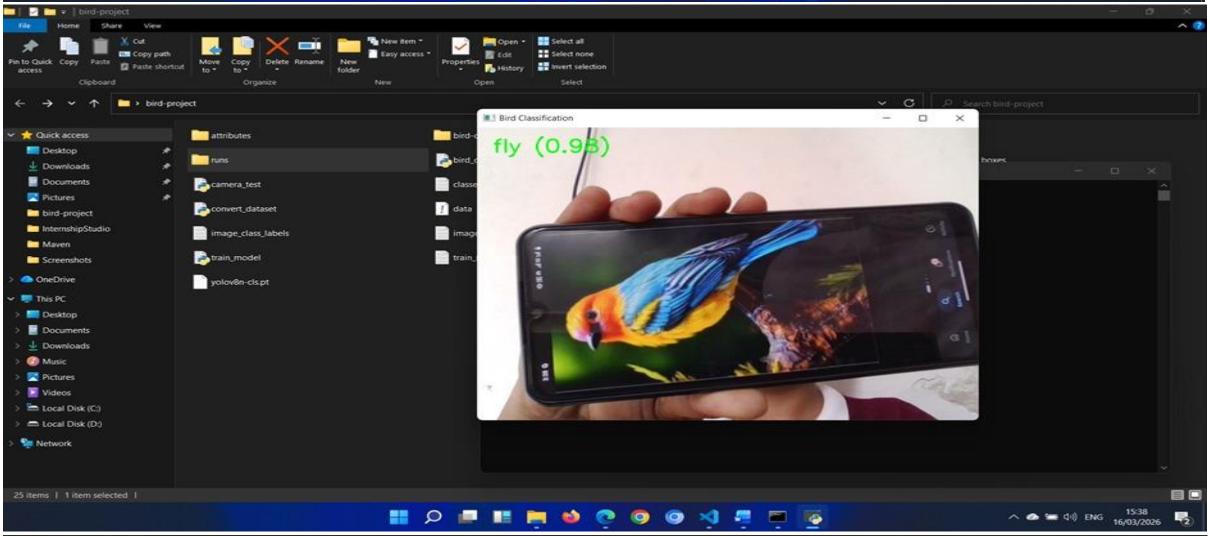
The model achieved strong performance on the validation dataset.

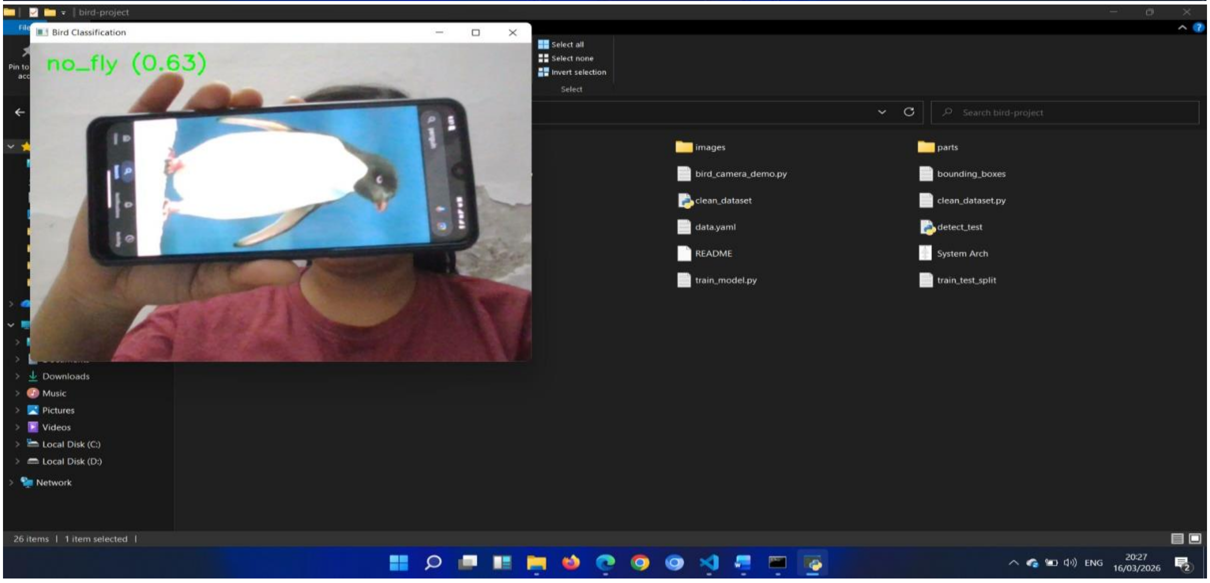
Metric	Value
Top-1 Accuracy	**0.985**
Top-5 Accuracy	**1.0**
Validation Images	467
Training Images	1501

Key Observations

- Training loss decreased steadily.
- Validation accuracy stabilized after ~20 epochs.
- Increasing epochs improved stability but gave minor accuracy gains.







Visualizations

Training generated multiple visualizations:

Training Metrics

- Train Loss Curve
- Validation Loss Curve
- Top-1 Accuracy
- Top-5 Accuracy

These graphs help analyze:

- Model learning progression
- Overfitting behavior
- Accuracy improvements across epochs

Example visualization:

Epoch vs Loss
Epoch vs Accuracy

These plots are automatically generated inside:

runs/classify/train/

Kaggle Submission

This project can be submitted on Kaggle as:

Computer Vision Classification Project

Submission includes:

- Trained YOLOv8 model
- Training scripts
- Dataset preprocessing code
- Performance metrics
- Visualizations

Project Structure

```
bird-project
|
├── bird-dataset
|   ├── train
|   |   ├── fly
|   |   ├── fly_swim
|   |   └── no_fly
|   |
|   └── val
```

```
|        ├── fly
|        ├── fly_swim
|       └── no_fly
|
├── convert_dataset.py
├── train_model.py
├── camera_test.py
├── detect_test.py
|
├── runs
|   └── classify
|
└── README.md
```

Future Work

Several improvements can enhance this project:

- Add **more bird behavior categories**
- Use **larger bird datasets**
- Train with **YOLOv8m or YOLOv8l models**
- Deploy model on **mobile or edge devices**
- Integrate **object detection + classification**
- Build a **web-based bird monitoring system**

Contributing

Contributions are welcome!

You can contribute by:

- Improving dataset quality
- Adding new bird categories
- Optimizing model performance
- Enhancing documentation

Please submit a **pull request** for improvements.

License

This project is licensed under the **Apache 2.0 License**.

Acknowledgements

Special thanks to:

- The **Ultralytics YOLOv8 team**
- **PyTorch** developers
- **Pixabay** for bird images
- **CUB-200-2011 dataset creators**
- Kaggle community for open datasets